

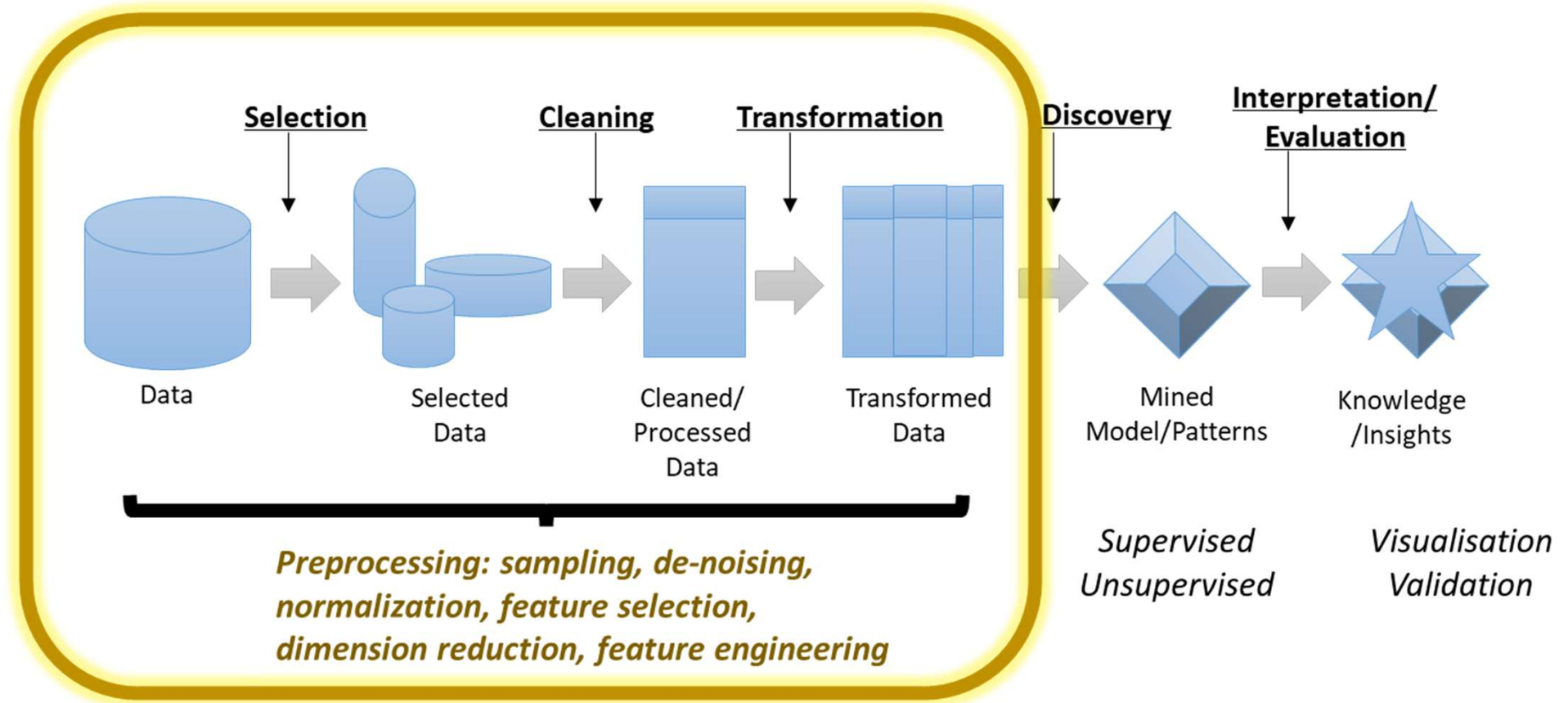
Postgraduate Studies in Business Analytics and AI

Data preprocessing

Jochen De Weerd

<https://bit.ly/session1-jochen>

This session



Session overview

1. Data selection and leakage
2. Exploratory data analysis
3. Data cleaning and transformation
4. Anomaly detection
5. Conclusion

Hands-on: four Colab notebooks

- All material in one GitHub repository: github.com/jochendeweerd/Session-day-1---Data-preprocessing
- Open a notebook in Google Colab (link and QR code on each hands-on slide), then File → Save a copy in Drive
- Every notebook has the same structure
 - Demo: I run it live, you follow along
 - Exercise A: change a parameter, rerun, interpret the result
 - Exercise B: write a few lines of code yourself
 - Solutions are in hidden cells (“Show code”)
- Section numbers in the notebooks match the slides (e.g. §3.4)

Notebook	Topic	Data
1	Data selection, prediction point and leakage	Telecom churn
2	Exploratory data analysis	House sales
3	Cleaning, transformation and pipelines	Telecom churn
4	Anomaly detection	Credit card transactions

1. Data selection and leakage

Our goal

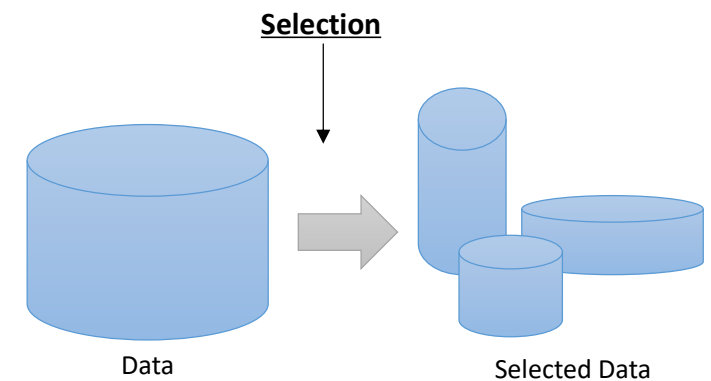
Transaction ID	Amount	Country	Timestamp	Merchant ID	Merchant Category (MCC)	Card type	Fraud
aA8zvGjlhO	211.99 €	Belgium	06/07/2020 01:31:02	n8mb	4511	VISA	No
jYJMgmV1Pw	545.09 €	Belgium	06/07/2020 01:45:33	3uty	3351	MASTERCARD	No
oDDL01Q8gK	8.99 €	Spain	06/07/2020 01:46:22	y5ir	4131	MASTERCARD	Yes
ykfQIZONVX	15.00 €	France	07/07/2020 23:32:59	yv0o	5311	VISA	No
n8tBNZE1vb	1 389.87 €	Belgium	07/07/2020 23:50:02	ucyo	3501	VISA	Yes
...	...	...	...	...	...	...	...

Our goal: a model-ready dataset

log amount	Country BE	Country ES	Country FR	Hour (sin)	Hour (cos)	Weekend	MCC (target enc.)	Card VISA	Merchant network risk	Fraud
5.36	1	0	0	0.39	0.92	0	0.004	1	0.00	0
6.30	1	0	0	0.44	0.90	0	0.006	0	0.02	0
2.20	0	1	0	0.45	0.89	0	0.021	0	0.31	1
2.71	0	0	1	-0.12	0.99	0	0.003	1	0.01	0
7.24	1	0	0	-0.04	1.00	0	0.012	1	0.18	1
...	...	...	...	...	...	...	...	...	...	...

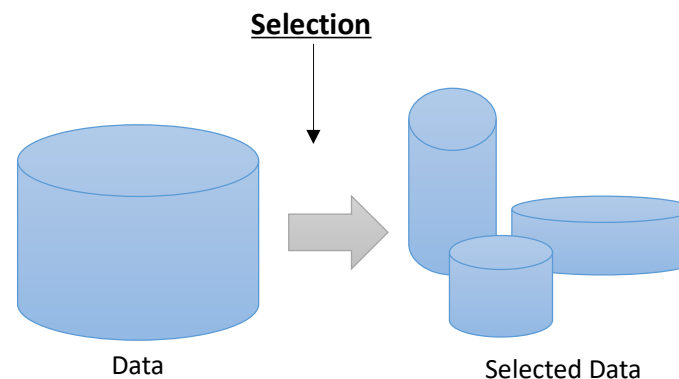
Data selection activities

- a) Assembling the dataset
- b) Defining the target and the prediction point
- c) Data leakage
- d) Setting aside a hold-out set
- e) Privacy and proxies



a) Assembling the dataset

- Before machine learning/AI algorithms can be used, a data set must be assembled
 - As machine learning algorithms can only uncover patterns actually present in the data, constructed data sets must be large enough to contain these patterns
 - But: concise enough so that algorithms run within an acceptable time limit
 - Usually resides in data marts, data warehouses, data lakes, ...



Assembling the dataset: sources and effort

- Constructing a dataset takes time and effort
 - Merging of data sources
 - Aggregation level (e.g. analysis at household vs. individual customer level)
 - Linking identifiers
 - Target variable definition
 - Data preprocessing including cleaning, transformation, feature engineering, etc.
- Where to find data?
 - Master data
 - Transactional data
 - External data
 - Open data
 - ...

Flattening the data



<https://app.datacamp.com/learn/skill-tracks/sql-fundamentals>

- Obtaining a (structured) tabular dataset requires:
 - Data extraction from source systems
 - Data flattening
- E.g. data from a relational database:

Transactions		
ID	Date	Amount
XWV	2/01/2015	52 €
XWV	6/02/2015	21 €
XWV	3/03/2015	13 €
BBC	17/02/2015	45 €
BBC	1/03/2015	75 €
VVQ	2/03/2015	56 €

Customer data		
ID	Age	Start date
XWV	31	1/01/2015
BBC	49	10/02/2015
VVQ	21	15/02/2015

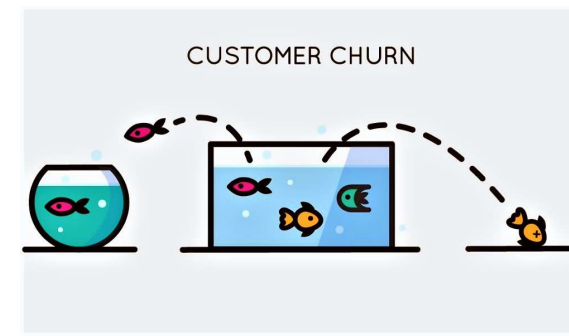
Non-normalized data table				
ID	Date	Amount	Age	Start date
XWV	2/01/2015	52 €	31	1/01/2015
XWV	6/02/2015	21 €	31	1/01/2015
XWV	3/03/2015	13 €	31	1/01/2015
BBC	17/02/2015	45 €	49	10/02/2015
BBC	1/03/2015	75 €	49	10/02/2015
VVQ	2/03/2015	56 €	21	15/02/2015

b) Target variable definition

- Predictive analytics (supervised learning): **target variable** (Y)
- Observed outcome

... or outcome to be defined?

- Customer churn prediction
 - Contractual vs. non-contractual setting
 - Active vs. passive churn
 - Expected and forced churn
- Fraud detection
 - Confirmed fraud cases by experts
 - Cases notified by customer
 - Rule system



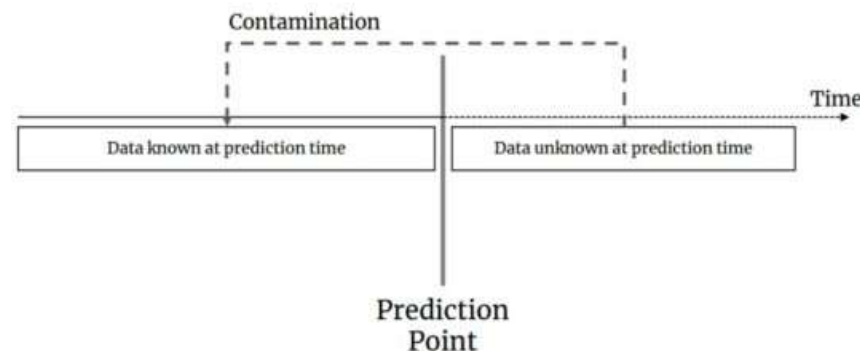
The prediction point

- Before selecting features, fix three moments in time
 - **Prediction point**: the moment at which the model is used (e.g. June 1)
 - **Observation window**: the period before the prediction point from which features are computed
 - **Outcome window**: the period after the prediction point in which the target is observed (e.g. churn during June)
- Every feature must be computable at the prediction point
- Also know the extraction date and how each variable was extracted
 - A “last call date” extracted on July 1 is not the last call before June 1
 - Many systems only keep the latest value, not the value at a past moment (snapshots)

c) Data leakage

When you want to predict a target variable y

- Make absolutely sure you're not cheating by including explanatory variables which are too "perfectly" correlated with the target (also called "data leakage")
- "Too good to be true" (overly optimistic expectations about model performance in production)
- Do you know this explanatory variable after the target outcome, or before?
 - Your finished model will not be able to look into the future!



<https://www.dataminingapps.com/2022/11/data-leakage/>

Data leakage: feature and instance leakage

- **Feature leakage**

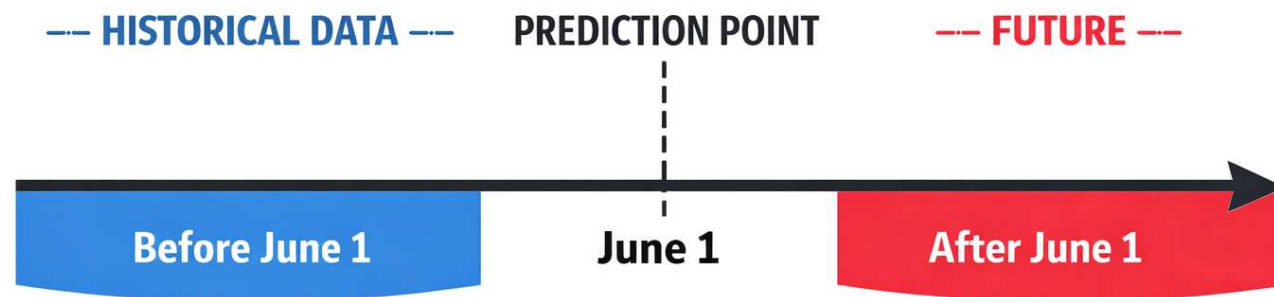
- Inclusion of features that are too correlated to the label
- E.g. feature “number of late payments” for a credit default prediction model
 - At the moment you need to make a prediction on whether a customer might default on a loan or not, the person does not have the loan yet
 - This might go unnoticed, e.g. when imputing “missing values” with zeros, but will lead to an under-estimate the probabilities
- E.g. unique identifier such as a customer ID, your ML-model can memorize those
- Can also arise due to a lack of snapshot-ability, i.e. not being able to retrieve the value of features for a past point in time (e.g. database records get updated)
 - E.g. number of calls into the helpdesk

- **Instance leakage**

- Improper sharing of information between rows of data
 - E.g. when applying preprocessing steps over multiple instances (e.g. imputation, see below)
- Duplicated or very similar instances in train and test set
 - E.g. 100,000 X-ray images of 20,000 patients, randomly splitting leads to leakage

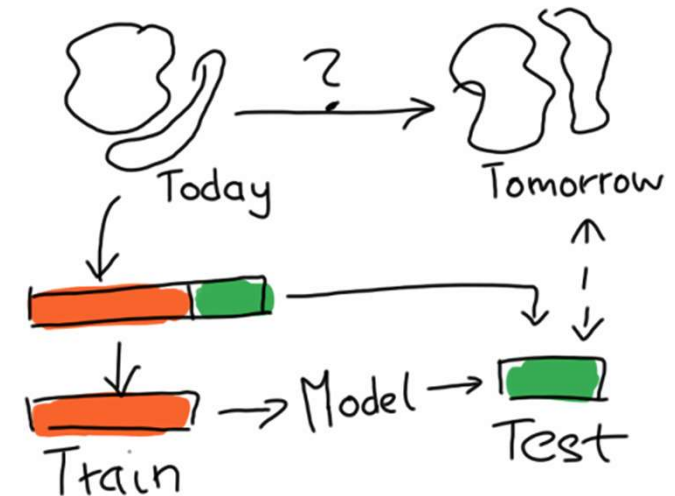
Data leakage is about time

- Leakage is not about suspicious variable names. Leakage is about time!
- Leakage = using information unavailable at prediction time



d) Setting aside a “hold-out” set

- Keep a piece of data separate to verify that your model works on unseen data
 - And to check that all your transformations remain applicable as well...
- Split before anything is learned from the data
 - Imputation values, scaling parameters, encodings, selected features: fit on the training data only
- Random or time-based split?
 - Temporal problems (churn, fraud): train on the past, test on the future
- (More on this, see: Supervised learning, next session day)



e) Privacy and proxies

- Only select what the business question needs (data minimisation, GDPR)
- Pseudonymise identifiers, and never use them as features (a model can memorise them)
- Watch out for proxies of sensitive attributes
 - Postal code, first name or language can act as a proxy for ethnicity, income or age
 - Leaving out the sensitive attribute itself does not remove the proxy effect
- Document where each variable comes from and when it was extracted

Hands-on 1: selection and leakage

- Notebook 1 – selection and leakage ([open in Colab](#))
- Case: churn prediction at a telecom operator, prediction point June 1, 2023
- Demo
 - Look at the raw extract and locate each column in time
 - Split first, then see how one leaky feature pushes the AUC to 1
- Exercise 1A: leakage detective – find and explain at least four more leaky columns
- Exercise 1B: repair a leaky feature (regional churn rate on the training data only)

Potential leaking variables

- **minutes_used_current_month**
 - If this reflects full June usage → leakage
 - June behavior is influenced by churn decision
- **last_call_date**
 - If extracted after churn → may include June calls
 - Must reflect last call *before* June 1
- **helpdesk_calls_3m**
 - Three months relative to what?
 - Safe: March–April–May
 - Leakage: window includes June
- **avg_monthly_bill**
 - Safe if based on historical billing (before June)
 - Leakage if June billing included
- **unpaid_invoices**
 - Must reflect state on June 1
 - If includes post-June invoices → leakage
- **contract_start_date**
 - Safe as raw date
 - But tenure must be computed as of June 1 (not extraction date)

Leakage detective: the verdict

Column	Single-column AUC	Verdict
minutes_used_current_month	1.00	Leak: June usage is the churn window itself
last_call_date	0.94	Leak: extracted July 1; active customers keep calling in June
retention_offer_sent	0.83	Leak: offers made when customers call to cancel, in June
contract_end_date	0.77	Leak: filled in when a contract is terminated; missingness leaks too
helpdesk_calls_3m	0.69	Mild leak: the three-month window includes June
minutes_used_last_month	0.69	No leak: a strong feature, known on June 1
region_churn_rate	0.67	Leak: computed on all customers, incl. own label and test set

Screening finds suspects, not culprits: a leak is proven by asking when and how a value was recorded.

Leakage danger

- Even if all timestamps precede the prediction point, leakage can occur if the feature value was determined using information that became available only after the prediction point, such as the absence or presence of events during the churn window.
- Simply filtering or truncating dates after 01/06 does not automatically fix leakage.
 - On June 1, you do not know whether May 25 will be the customer's last-ever call.
 - That information only exists after observing June behaviour
- Many information systems don't allow for proper snapshotting because they will only retain the latest status/value/date
 - You should factor in the data generating process!

2. Exploratory data analysis

From data selection to exploratory data analysis

- Data selection gives you a candidate dataset; it does not tell you whether that dataset can answer the business question
- EDA is the first check of the choices made during selection
 - Is the target what we think it is? (definition, rate, distribution)
 - Are the features what we think they are? (units, ranges, dates after the prediction point)
 - Is the data representative of the population on which the model will be used?
- EDA is also the first leakage detector: a variable that separates the target almost perfectly deserves suspicion
- EDA findings decide what happens next in cleaning and transformation
- Explore the training data: decisions based on the test data leak information as well

Exploratory Data Analysis (EDA)

“The greatest value of a picture is when it forces us to notice what we never expected to see.”

John Tukey



Why EDA is not optional

- EDA reduces modeling uncertainty
- EDA reveals structure and risk
- EDA shapes preprocessing decisions
- EDA prepares interpretation and deployment

➔ If EDA did not change your thinking, it was superficial.

EDA objectives

- 1) Get a feel of the data, describe the data, look at a sample of data like first and last rows
- 2) Test business assumptions, generate hypotheses for further analysis
- 3) Take a deeper look into the data by querying or indexing the data
- 4) Identify features of interest
- 5) Recognize the challenges posed by data - missing values, outliers
- 6) Discover patterns in the data, prepare the data for modelling
- 7) Develop an intuition for interpreting the results of future modeling

What to look at

- Structure: rows, columns, data types, identifiers, duplicates
- Target: distribution, skewness, class balance
- Missing values: how much, where, and why (structural or random?)
- Univariate distributions: outliers and impossible values
- Relationships: target versus features, redundancy between features
- Time: trends, seasonality, changes in definitions
- Each finding should lead to a decision
 - Keep, drop, transform, impute, or ask the data owner

From EDA findings to decisions

What EDA shows	Why it matters	Next step
Distribution, rate and definition of the target	Is the question well operationalised? Skewed or rare target?	Target definition, target transformation, evaluation metric
Values or dates after the prediction point; near-perfect predictors	Possible leakage	Drop or recompute the feature
Missing, impossible and inconsistent values	Why are they missing or wrong?	Cleaning rules, imputation, missing indicators
Skewed distributions and extreme values	Influence on sensitive models, or the signal itself	Transformations, outlier treatment, anomaly detection
Rare categories or many levels	Unstable estimates, many dummy variables	Grouping and encoding
Strongly related features	Redundancy and multicollinearity	Feature selection or reduction

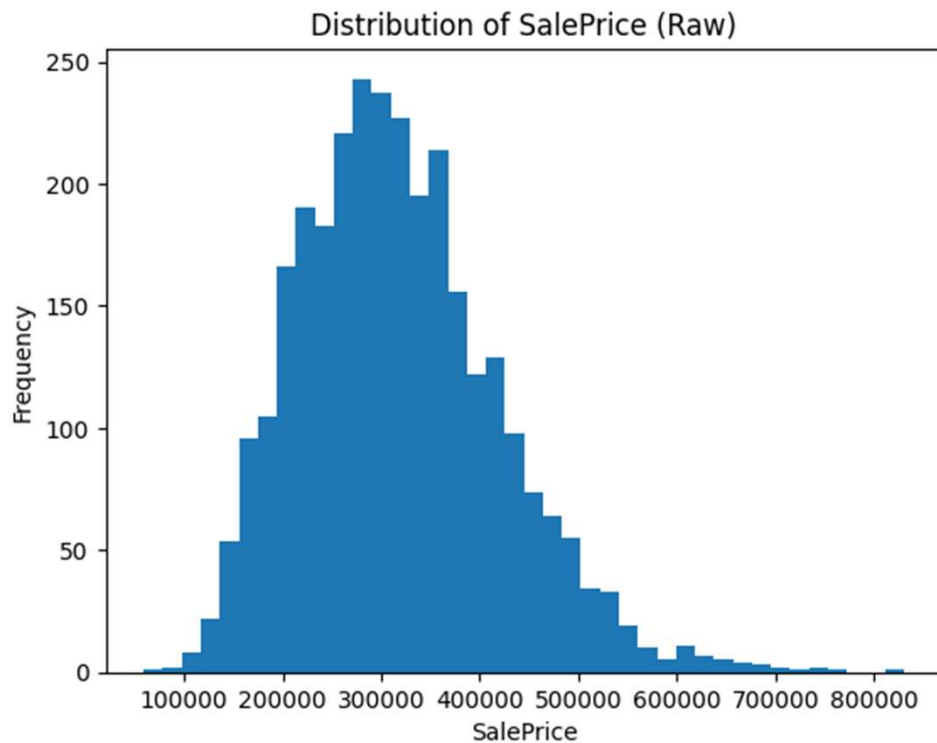
Automated profiling and AI assistants

- Profiling tools give a first overview in one line
 - skrub TableReport, ydata-profiling, missingno (patterns of missing values)
- AI assistants can write an EDA for you (e.g. Gemini in Google Colab)
- Use them to start, not to conclude
 - Check what they assumed: did they split before imputing? Did they notice structural missingness?
 - Only upload data you are allowed to share
- Plotting libraries: pandas, matplotlib, seaborn, plotly

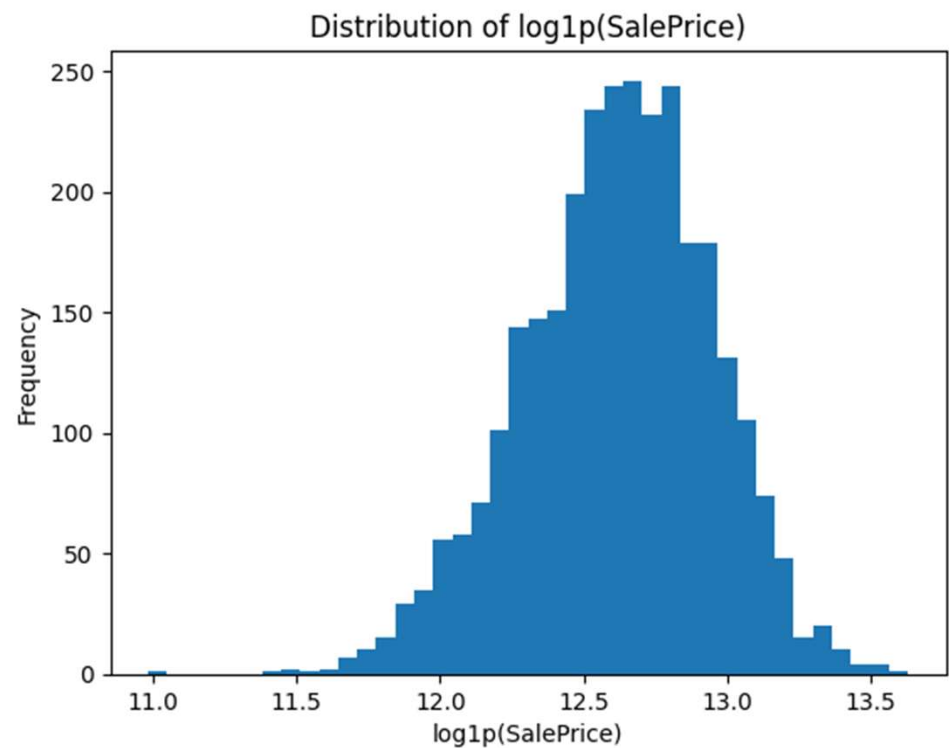
Hands-on 2: exploratory data analysis

- Notebook 2 – exploratory data analysis ([open in Colab](#))
- Case: 3,000 house sales, target SalePrice
- Demo
 - Profiling, target distribution, missing values, impossible values
 - Relationships with the target and redundancy
- Exercise 2A: improve a plot (trustworthy, actionable, elegant)
- Exercise 2B: let an AI assistant do the EDA, then review it
- End with your list of findings and decisions (§2.9)

Target analysis – SalePrice distribution



Skewness (raw SalePrice): ≈ 0.656



Skewness ($\log_{10}(\text{SalePrice})$): ≈ -0.307

Modeling implications

- **If using linear regression**

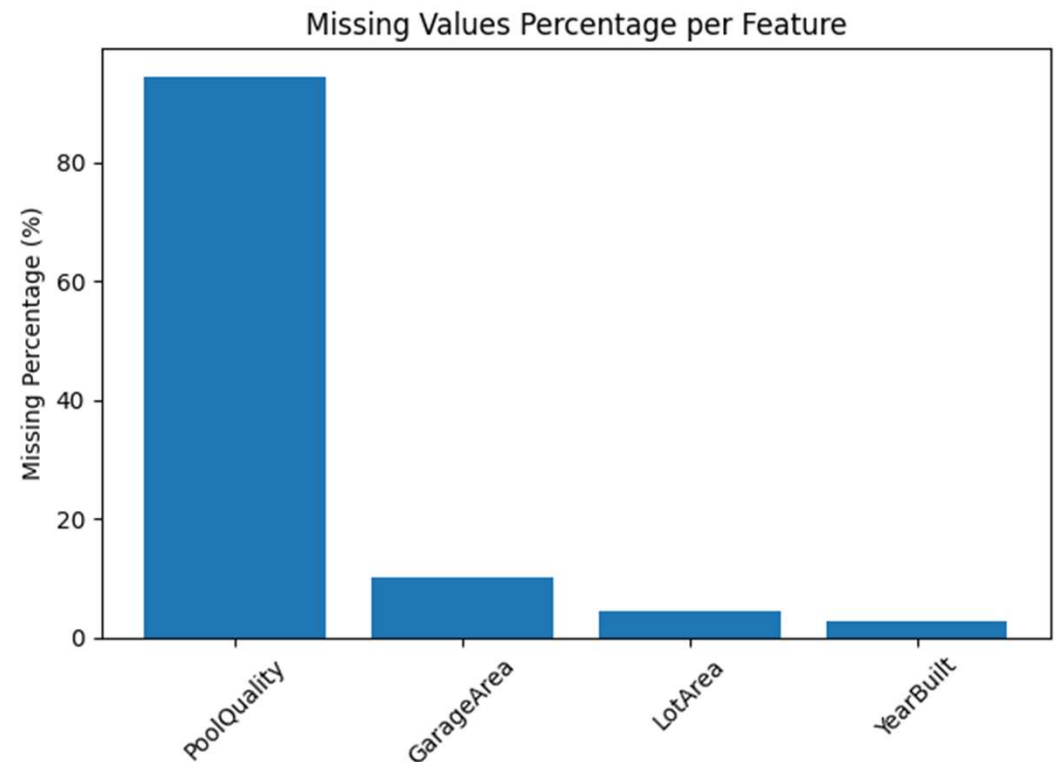
- Log transformation may:
 - Improve residual normality
 - Stabilize variance
 - Improve model fit
- But:
 - Coefficients become multiplicative
 - Interpretation changes

- **If using tree-based models**

- No normality assumption
- Skewness typically not problematic
- Transformation often unnecessary

Missing values in the housing data

- Missingness in the dataset
 - PoolQuality: 94.47%
 - GarageArea: 10.03%
 - LotArea: 4.43%
 - YearBuilt: 2.70%
- Observation: Missingness is highly concentrated in one variable.

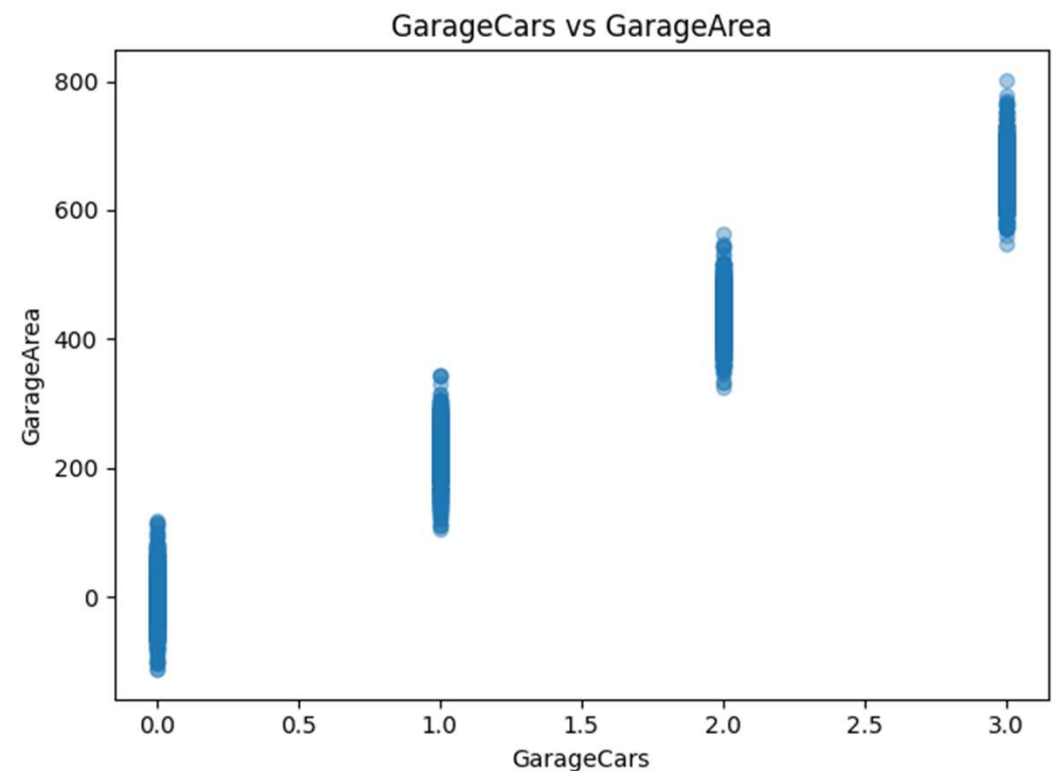


Pool Quality: drop or keep?

- Facts
 - 94.47% missing
 - Only relevant for houses that have a pool
 - Key question:
 - Is this random missingness or structural missingness?
- Interpretation
 - If a house has no pool, PoolQuality is logically missing
 - This is structural
 - Missing here encodes information
- Strong reasoning
 - Do not drop blindly
 - Encode missing as “No Pool”
 - Preserve signal

Redundancy

- Candidate pair: GarageCars vs GarageArea
- Correlation: 0.987
- This is almost perfect linear correlation
- Interpretation: both variables capture nearly identical information

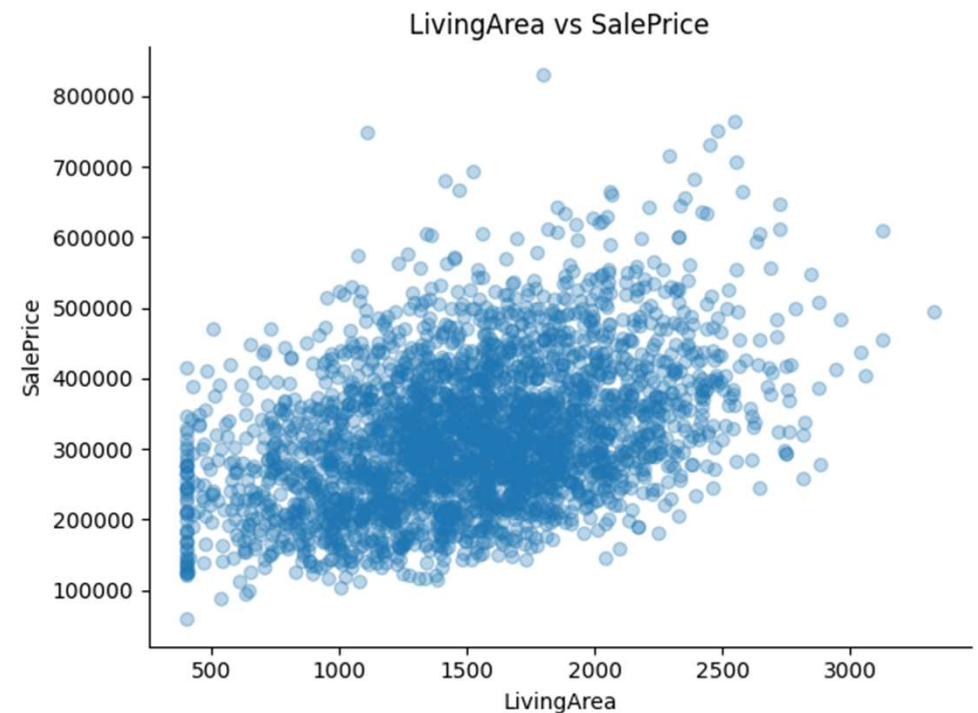


Correlation is not enough

- Important caution:
 - Correlation captures only linear relationships
 - Weak correlation does not imply irrelevance
 - Tree-based models may capture nonlinear effects
- Question: Would you drop variables purely based on low correlation?

Visualization quality

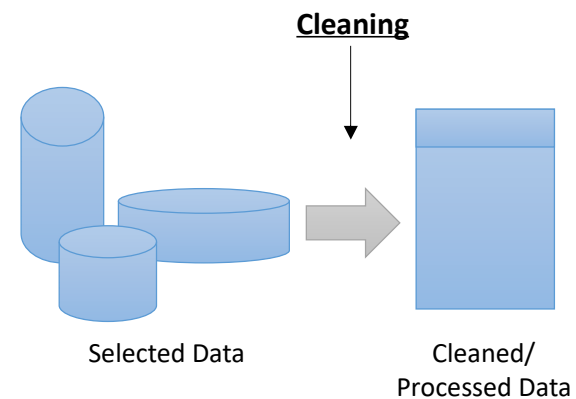
- What changed?
 - Reduced visual clutter
 - Removed unnecessary frame lines
 - Reduced opacity for density insight
 - Clearer emphasis on relationship
- Same data, only presentation changed



3. Data cleaning and transformation

Data cleaning activities

- a) Basic consistency
- b) Dealing with missing values
- c) Outliers



a) Basic consistency

- Detect errors/duplications in the data and rectify them if possible:
 - K.U.L., KU Leuven, K.U.Leuven, KULeuven...
 - 3000, LEU-3000, 3000 Leuven
- Convert formats to a common representation:
 - Male, Man => M
 - True, yes, ok => 1
 - 2020-03-15 and 15/03/2020
- Remove “future variables” (see leakage)
- Write consistency rules as code, so that they also run on new data
 - Data validation libraries: pandera, Great Expectations
- Recall again: the data warehouse – if you’re (very) lucky this is already fixed for you

b) Dealing with missing values

Instance	Outlook	Temp	Humidity	Windy	Play Golf?
1	rainy	26	high	no	no
2	rainy	24	high	yes	no
3		25	high	no	yes
4	sunny		high	no	yes
5	sunny			no	yes
6	sunny	17	normal		no
7	overcast	16	normal	yes	yes
8	rainy	19		no	no
9			normal	no	yes
10	sunny	14	normal		yes
11		15	normal	yes	yes
12	overcast	14	high	yes	yes
13	overcast	28		no	yes
14	sunny	27	high	yes	no

How to deal with missing values?

Dealing with missing values

- Most techniques cannot deal with missing values out of the box
 - There are some exceptions, decision trees for instance can be modified to work with three way splits, e.g. AGE < 20? Yes, No, NA
- Why is it missing? Completely at random (MCAR), at random (MAR) or not at random (MNAR)?
 - Non applicable versus non disclosed
- Detection is easy (see EDA), understanding why is not
- Main treatment options:
 - Delete row/column: e.g. when too many missing values exist for that row/column
 - Replace (impute): estimate missing value, replace e.g. by mean, median or mode
 - Keep: include a separate missing indicator variable

Delete, replace, or keep

Instance	Outlook	Temp	Temp Missing?	Humidity	Windy	Play Golf?
1	rainy	26	no	high	no	no
2	rainy	24	no	high	yes	no
3	sunny (mode)	25	no	high	no	yes
4	sunny	20 (mean)	yes	high	no	yes
5	sunny		no		no	yes
6	sunny	17	no	normal	no (mode)	no
7	overcast	16	no	normal	yes	yes
8	rainy	19	no	unknown (separate value)	no	no
9	sunny (mode)		no	normal	no	yes
10	sunny	14	no	normal	no (mode)	yes
11	sunny (mode)	15	no	normal	yes	yes
12	overcast	14	no	high	yes	yes
13	overcast	28	no	unknown (separate value)	no	yes
14	sunny	27	no	high	yes	no

Delete, replace, or keep

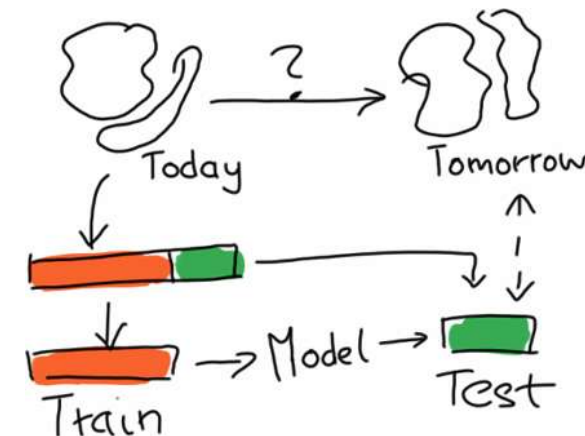
- Approaches can be combined, best practices:
 - Option 1
 - Delete rows with many missing values
 - Impute with median (continuous) and mode (categorical)
 - Add separate columns indicating whether original was missing or not
 - Option 2
 - Delete rows with many missing values
 - Impute with median (continuous) and separate category (categorical)
 - For continuous: add separate columns indicating whether original was missing or not
- In scikit-learn: `SimpleImputer(strategy="median", add_indicator=True)`

More advanced analysis and imputation

- Check if variables are missing at random or whether they depend on other variables
- In some cases, a predictive model could be build just for imputing the missing values
- Or alternative advanced techniques can be used: e.g. nearest neighbour imputation
 - In scikit-learn: KNNImputer, IterativeImputer
- Some models handle missing values natively (e.g. gradient boosting)

Model testing and deployment

- Take care: once your model is built, and you want to predict new, unseen observations
 - This data can contain missing variables as well
 - Don't impute with a new average calculated over this bunch of new data!
 - Does not necessarily follow the same distribution
 - If you imputed missing values during the construction of your model for that feature, then replace with the same average / median / mode / value you used during construction (thus on the training data)
 - If this is the first time you encounter a feature with a missing value here:
 - Replace with the mean / median / mode / ... of the feature as you saw it in the original training set (to not introduce shift in model)
- **Everything you do over multiple instances becomes part of the full model pipeline!**



Missing values

- Only one variable contains missing values:
 - **avg_monthly_bill**
- **Key question:** Why is it missing?
 - Possible business explanations:
 - Prepaid customers do not receive monthly bills
 - Very new customers have not yet been billed
 - Data extraction artifact
- **Critical insight**
 - Missingness is likely **not random**
 - If missing correlates with:
 - contract_type
 - contract_start_date
 - or even churn
 - Then missingness itself contains signal
- Deleting rows or blindly imputing may destroy predictive information

Correct treatment

Step 1 – Business logic first

- If prepaid customers have no bill:
 - Missing is **structural**, not noise
 - Consider:
 - Create indicator bill_missing
 - Set bill = 0 for prepaid

Step 2 – Avoid leakage

- Wrong:
 - Compute global mean bill on full dataset
 - Impute before splitting
- Correct:
 - Split at **2023-06-01**
 - Compute imputation value using training data only
 - Apply same value to test set

Core takeaway

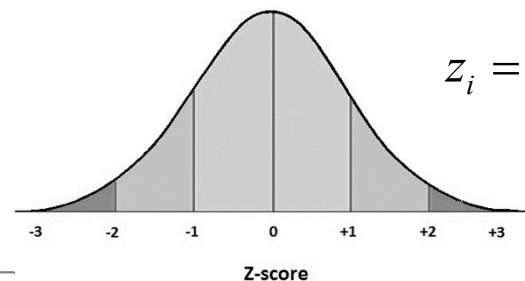
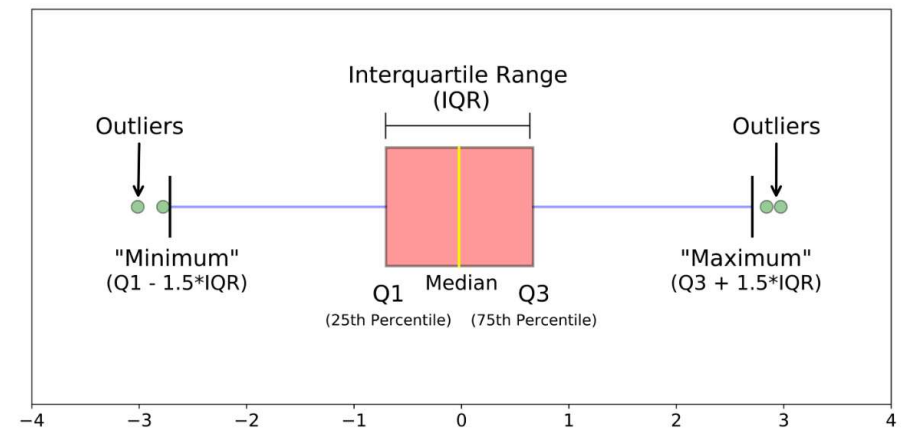
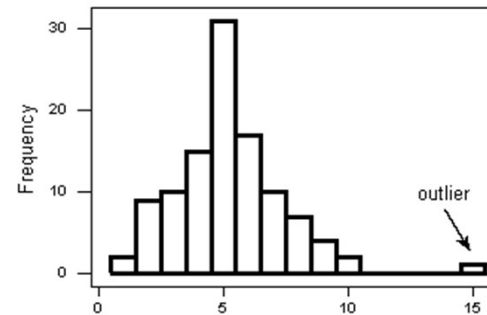
- Missing values are not a technical issue.
- They are a **business signal** and a potential **leakage trap**.

c) Outliers

- Outlier = extreme observation (continuous variable)
 - Examples
 - Age = 241
 - Income = 1 241 471 euro
 - Temperature = 51 degrees Celsius
- Valid observations: just very extreme outliers
- Invalid observations: due to errors, measuring mistakes, data entry error
- Difference between outlier detection and treatment

Outlier detection

- Histograms
- Box-plots and the IQR
- Z-scores
- Anomaly detection (see Session on Fraud Analytics)

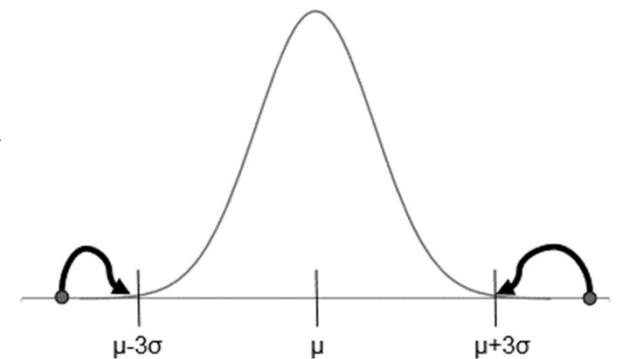


$$z_i = \frac{x_i - \mu}{\sigma}$$

Any z-score greater than 3 or less than -3 is considered to be an outlier.

Outlier treatment

- For invalid outliers: just consider them as missing values
- For valid ones: treatment depends on technique you'll use
 - Some techniques are very robust to outliers (e.g. decision trees)
 - Other techniques are not (e.g. regression)
- Truncate/cap, e.g. based on z-score:
 - Replace all variable values with z-scores > 3 with mean $+ 3 * \text{stdev}$
 - Replace all variable values with z-scores < -3 with mean $- 3 * \text{stdev}$
 - More robust: cap at percentiles of the training data (e.g. 1% and 99%)
- Bin them = put in separate group (categorize)



Treatment strategy: think before cleaning

Step 1 — Invalid outliers

- If clearly impossible:
 - Negative minutes
 - 500 unpaid invoices
 - Billing amounts outside product limits
- Treat as missing
→ Then follow missing value pipeline

Step 2 — Valid but extreme

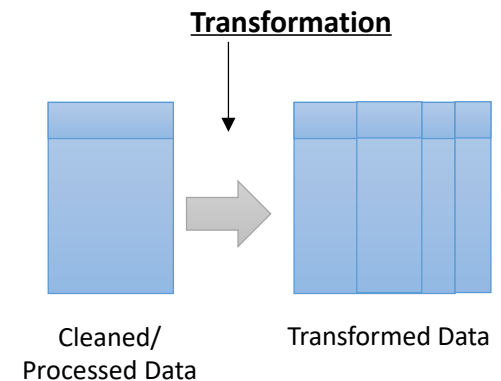
- Example:
 - Sudden drop to near 0 minutes
 - Explosion in helpdesk calls
- In churn prediction, these may be **strong signals**.
- Removing them:
 - Can destroy predictive power
 - Especially dangerous in churn or fraud tasks (they may be the signal you are looking for)

Model sensitivity matters

- Linear / regression models → sensitive to outliers
- Decision trees → largely robust

Overview

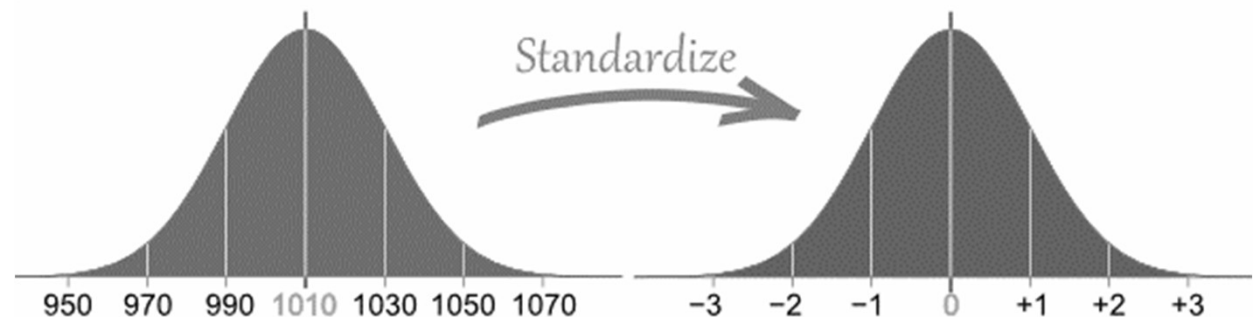
- a) Scaling and power transformations
- b) Categorization and encodings
- c) Feature engineering
- d) Feature selection and reduction
- e) Putting it together: pipelines



a) Standardization

- Standardization: rescale a feature to zero mean and a standard deviation of one (z-score)
 - The shape of the distribution does not change: skewed data stays skewed
 - Good for domains with Gaussian-distributed data: socio-demographics, medical
 - Not effective for some techniques! E.g. most decision tree learners only rely on ordering of data, transformation does nothing to change this, so don't bother
 - Good or required for other techniques (like k-nearest neighbour classification)

$$x_{new} = \frac{x - \mu}{\sigma}$$



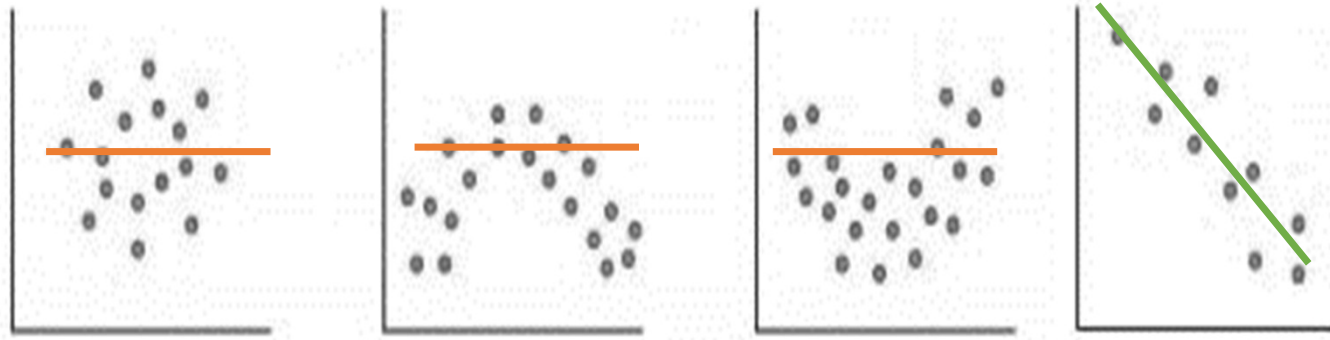
Normalization

- Also called **feature scaling**
- I.e. rescale so features fall in $[0, 1]$ or $[-1, 1]$
- Sensitive to outliers: one extreme value squeezes all other values together
→ robust scaling (median and interquartile range)

$$x_{new} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

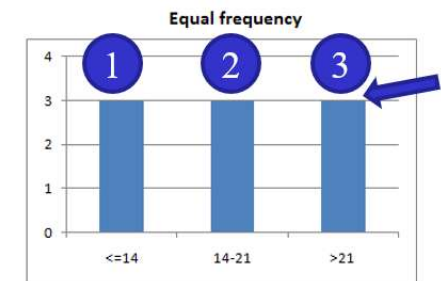
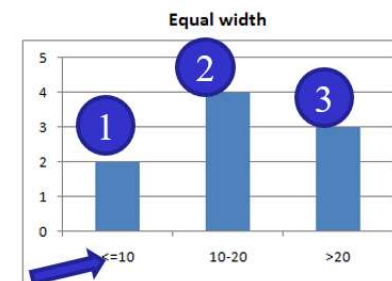
Other common transformations

- Logarithms, square roots, etc.
 - Mostly done to make relationships more linear, or distributions more Gaussian
 - Despite the uptake of advanced and non-linear machine learning techniques, still beneficial for many techniques
 - Box-Cox, Yeo-Johnson and other “power transformations”



b) Categorization

- Continuous → nominal
 - **Binning**: group ranges of continuous features into categories
 - In many cases: driven by domain knowledge
 - Can be a very powerful way to guide the predictive algorithm
 - Oftentimes driven by domain knowledge
 - Equal width/interval binning vs. equal frequency binning
- Nominal → reduced nominal
 - **Grouping**: grouping multiple category levels together: in case you have many levels (e.g. long tail of infrequent countries)
 - High cardinality categoricals (e.g. city/postal code, job titles, etc.)



Dummy variables and encodings

- What if your algorithm cannot deal with categorical features?
 - E.g. regression, “distance”-based techniques, neural networks, etc.
- Nominal → continuous
 - Transform categoricals into numerical features
 - Dummy variables or “one-hot” encoding
 - Mainly used in regression (cannot deal with categoricals directly)

Dummy variables

- Converting categorical variables to numeric ones:
 - A dummy variable or indicator variable is an artificial variable created to represent an attribute with two or more distinct categories/levels
 - Mainly used in regression: need to treat all variables as numerical
- E.g. weather = sunny, raining, overcast

Convert to:

weather_sunny = 0/1

weather_raining = 0/1

weather_overcast = 0/1



One can be left out

More advanced approaches

- Domain-knowledge driven grouping (e.g. NACE codes)
- Odds grouping
- **Target encoding and WoE encoding**
- Categorical embeddings
- Build a simple decision tree base on one categorical/continuous variable an target
- 1-dimensional k-means clustering on continuous variable to suggest groups
- **NEVER:** integer encoding if your variable is not ordinal!

Weights of Evidence encoding (WoE)

- WoE measures the “strength” of a grouping to separate good and bad
- Weights of evidence variables can be defined as follows:

$$WoE_{cat} = \ln\left(\frac{p_{c1,cat}}{p_{c2,cat}}\right)$$

$p_{c1,cat}$ = number of instances with class 1 in category / number of instance with class 1 (total)

$p_{c2,cat}$ = number of instances with class 2 in category / number of instance with class 2 (total)

- If $p_{c1,cat} > p_{c2,cat}$,cat then $WoE > 0$
- If $p_{c1,cat} < p_{c2,cat}$,cat then $WoE < 0$

Target encoding: use the target, avoid the leak

- Replace each category by the (smoothed) average target in that category
 - Same idea as WoE; works for classification and regression
 - Smoothing pulls rare categories towards the overall mean
- Danger: the encoding uses the target
 - Computed on all data, a row's own label leaks into its own feature, especially for rare categories
- Solution: fit on the training data only, with cross-fitting (out-of-fold encoding)
 - In scikit-learn: TargetEncoder does this internally

c) Feature engineering

- With feature engineering you want to transform a set of data set variables into features that help analytical models to achieve better performance (accuracy, interpretability, ...)
- Most modelling techniques are quite “dumb”
 - E.g. date of birth vs. age
 - E.g. timestamp vs. week/weekend, day of the week, time of the day, ...
- Of course, you need to be careful to avoid biasing data! Creating leakage is always a danger!

Feature engineering

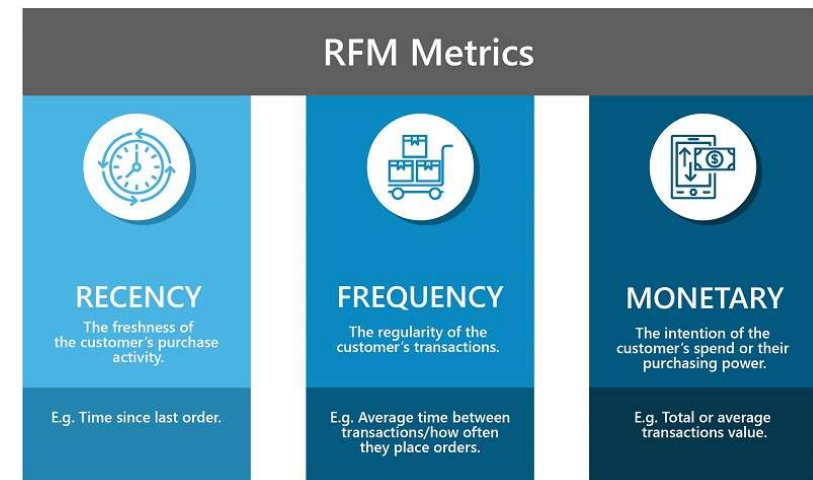
- Common approaches:
 - Delta's
 - Trends
 - Windows
- Often also domain driven

Conversion from distance
(numeric) to meaningful
category (walkable?)

Lat. 1	Long 1.	Lat. 2	Long. 2	Distance (km)	Can Walk?
48.871507	2.354350	48.872111	2.354933	2	Yes
48.872111	2.354933	44.597422	-123.248367	9059	No
48.872232	2.354211	48.872111	2.354933	5	Yes
44.597422	-123.248367	48.872232	2.354211	9056	No

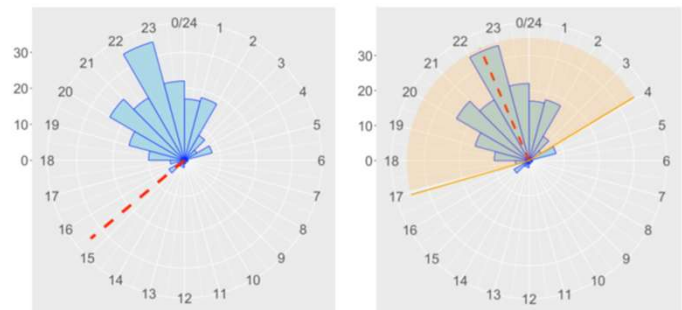
RFM features

- Framework proposed by Cullinan in 1977
- Originally in the marketing domain (customer segmentation)
- Many different “operationalizations”
 - Recency (days since last interaction/purchase)
 - Frequency over a certain period (last week, last month, last year)
 - Average, minimum, maximum monetary value over the last week, year, month, etc.
 - For different product types
- Typically, you would code this yourself (e.g., <https://www.kaggle.com/code/regivm/rfm-analysis-tutorial>), but some people have developed dedicated libraries (e.g., <https://pypi.org/project/rfm/>)



Datetime features

- Capture information about the time aspect by meaningful features
- Typically, don't use a date field directly as a feature, but derive and use
 - Time since date (in days, hours, ...)
 - Quarter (1-4), month of the year (1-12), week (1-52), day of the month (1-31)
 - Day of the week (Mon-Sun), weekend or not (0, 1), holiday or not (0, 1)
- Dealing with time can be tricky
 - 00:00 = 24:00
 - 23:00 > 01:00?
- Do not use the arithmetic mean to compute an average timestamp
- Encode cyclical features with a sine and a cosine (hour of the day, day of the week, month)
 - For averages of times, use circular statistics (e.g. the von Mises distribution)



Feature engineering relative to the prediction point

- Compute every feature relative to the prediction point, not to the extraction date
 - $\text{tenure_days} = \text{prediction point} - \text{contract start date}$
- From levels to behaviour
 - $\text{usage_ratio} = \text{minutes last month} / \text{minutes two months ago}$
 - $\text{usage_drop} = 1$ if the ratio is below 0.8
 - $\text{is_new_customer} = 1$ if tenure is below 180 days
- Domain grouping: postal code (280 levels) $\rightarrow$ first two digits (≈ 80 areas)
- Row-wise rules like these learn nothing from the data: they may be computed before the split, but must run identically in production

Learned representations: embeddings

- An embedding maps a category, a text or an image to a dense numeric vector, learned by a neural network
 - Similar items get similar vectors
- Categorical (entity) embeddings
 - Learned while training a neural network on the prediction task
 - Alternative to one-hot encoding for high-cardinality variables (merchant, product, postal code)
- Text embeddings from pretrained language models
 - Complaint texts, product descriptions, job titles → a few hundred numeric features
 - E.g. sentence-transformers, skrub TextEncoder, or an embedding API
- Embeddings can feed any model (logistic regression, gradient boosting, ...)
 - But: harder to interpret, and the embedding model becomes part of the pipeline

Network features and graph embeddings

- Many business datasets are networks
 - Cards and merchants, customers who call each other, companies and their directors
- Hand-crafted network features
 - Degree (e.g. number of distinct merchants per card)
 - Exposure: share of fraud or churn among the neighbours
 - Shared devices, addresses or bank accounts
- Learned network features
 - Node embeddings (DeepWalk, node2vec) or graph neural networks
- As always: build the network only from information available at the prediction point
- Example in fraud detection: Van Belle, Baesens & De Weerd (2022), CATCHM, Decision Support Systems

Large language models in feature engineering

- Turn unstructured text into structured features
 - Extract fields (reason for a complaint, product mentioned), classify topic or sentiment
- Clean and standardise messy values
 - “K.U.L.”, “KU Leuven”, “Katholieke Universiteit Leuven” → one entity
- Suggest candidate features from a description of the data and the business problem
 - E.g. CAAFE (Hollmann, Müller & Hutter, 2023)
- Watch out
 - Hallucinations: validate the output on a sample
 - Reproducibility: fix the model version and settings, store the outputs
 - Privacy and leakage: what does the model see, and where is the data sent?
 - Cost and latency when the feature is needed for every new record

Preprocessing tooling in Python

- **scikit-learn**
 - Pipeline, ColumnTransformer, SimpleImputer, TargetEncoder, set_output(transform="pandas")
- **skrub**
 - TableReport, TableVectorizer, tabular_pipeline (a strong default pipeline for a dataframe)
 - <https://skrub-data.org>
- **Feature-engine**
 - Transformers to engineer and select features, with fit() and transform()
 - <https://feature-engine.trainindata.com>
- **Featuretools**
 - Automatically creates features from temporal and relational datasets



d) Feature selection

- Creating many features is nice, but...
 - Curse of dimensionality: lots of features will make many techniques go haywire
 - Need to have a method to throw out features without any or very weak predictive power
 - E.g.: remove features with zero variance (same level everywhere)
- Three main approaches
 - Filtering
 - Wrapping
 - Embedded methods

Filtering (a.k.a. variable ranking)

- Independent of the model/machine learning technique
- Throw out weak features
- Variations:
 - Variance threshold-based
 - If variance is too low, remove
 - Chi-squared test: goodness-of-fit test or chi-square test for independence
 - Check if target and variable are independent: if they are, throw out variable
 - Based on P-value
 - Does not take into account interaction effects (univariate test)
 - Information gain / mutual information (also captures non-linear relations)
 - Typically used for categorical variables

Wrapping

- Wrapper methods evaluate subsets of features making use of the learned model for each subset
- Approaches:
 - Exhaustive search (impossible for large number of predictors)
 - Greedy strategies
 - Forward selection: starting from an empty model and add most important variables step by step
 - Backward elimination: starting from a full model and eliminate unimportant features step by step
 - Step-wise
- In scikit-learn: SequentialFeatureSelector, RFE (recursive feature elimination)

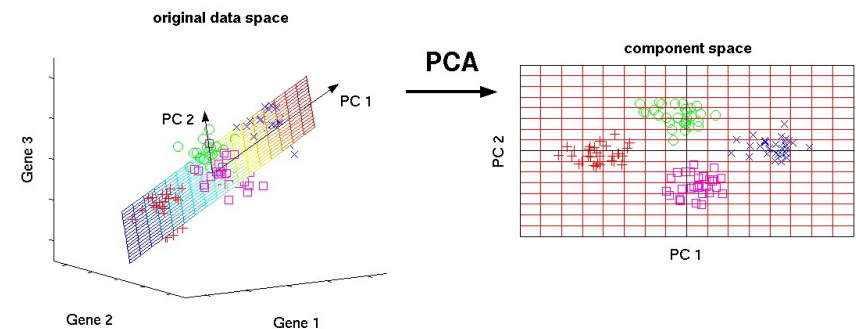
<https://towardsdatascience.com/feature-selection-for-machine-learning-in-python-wrapper-methods-2b5e27d2db31>

Embedded methods

- Selection happens while the model is trained
 - L1 regularisation (Lasso): coefficients of weak features are shrunk to exactly zero
 - Tree-based models: features that are never used in a split get zero importance
- Cheaper than wrapping, but tied to one type of model
- Selection is a learned step as well: select on the training data, inside cross-validation

Dimensionality reduction: PCA

- Principal components analysis (PCA)
 - Convert possibly correlated variables into a set of linearly uncorrelated variables called principal components
 - The number of principal components is less than or equal to the number of original variables
 - Requires numerical, scaled input (for categorical data: e.g. multiple correspondence analysis)
 - Important: interpretability might become more difficult
 - Example: V1–V28 in the credit card data of Notebook 4 are principal components (for confidentiality)
- Non-linear techniques (t-SNE, UMAP)



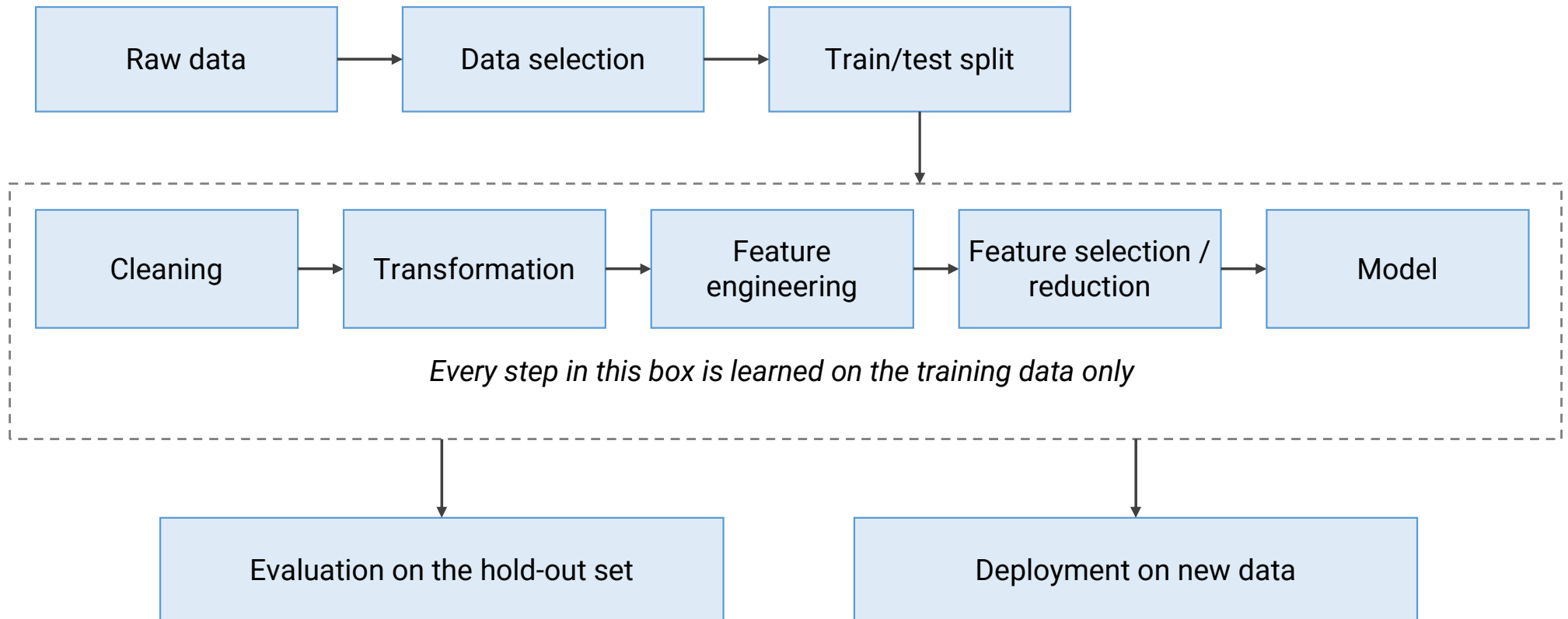
<http://setosa.io/ev/principal-component-analysis/>

e) Putting it together: pipelines

- Every learned step is part of the model: fit on the training data, apply unchanged to test and production data
- In scikit-learn: Pipeline + ColumnTransformer
- Cross-validate the whole pipeline, not only the model

```
preprocess = ColumnTransformer([
    ("num", make_pipeline(SimpleImputer(strategy="median", add_indicator=True),
                          StandardScaler()), numeric_columns),
    ("contract", OneHotEncoder(), ["contract_type"]),
    ("region", TargetEncoder(), ["postal_code"])]])
model = make_pipeline(preprocess, LogisticRegression())
model.fit(X_train, y_train)          # everything is learned on the training data
model.predict_proba(X_new)          # the same steps are applied in production
```

The full preprocessing pipeline



How much preprocessing do modern models still need?

- Gradient boosting (XGBoost, LightGBM, CatBoost, HistGradientBoosting)
 - Handles missing values and categorical features natively, needs no scaling
- Tabular foundation models (e.g. TabPFN, TabICL) take a raw table as input
- Automated baselines (e.g. skrub tabular_pipeline) build a reasonable pipeline in one line
- So mechanical preprocessing is increasingly automated...
- ... but no tool knows your prediction point, your target definition or which information is allowed
 - Still needed: linear and distance-based models, interpretability, domain features

Hands-on 3: cleaning, transformation and pipelines

- Notebook 3 – cleaning, transformation and pipelines ([open in Colab](#))
- Case: the churn data of Notebook 1, without the leaky columns
- Demo
 - Consistency rules, split, structural missingness, invalid vs. valid outliers
 - Features relative to the prediction point, one pipeline fitted on train
 - Leakage through target encoding; modern models vs. an automated baseline
- Exercise 3A: compare preprocessing choices
- Exercise 3B: write your own learned transformer

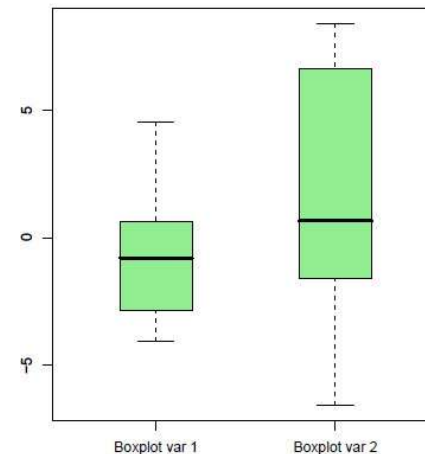
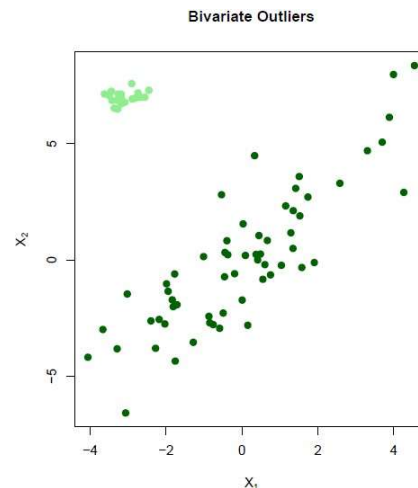
4. Anomaly detection

From outliers to anomalies

- In data cleaning, an outlier is a problem to fix
- In anomaly detection, the outlier is the signal
 - Fraud, intrusions, machine failures, pre-churn behaviour
- Unsupervised: describe normal behaviour, flag what deviates from it
 - No labels needed, and new patterns can be found
- An anomaly is not a fraud: alerts need follow-up and validation
- Combining anomaly detection with supervised models: see the session on fraud analytics

Univariate vs. multivariate outlier

- Former methods all focus on univariate outliers
- In the multivariate setting, outliers can NOT always be detected by simply applying outlier detection rules to each variable separately
- We can only detect such outliers by correctly estimating the **covariance structure**



Multivariate outlier detection

- Use of the Mahalanobis distance
 - Distance measure that accounts for the dispersion in each variable
- Clustering-based
 - Eg. kNN distance or hierarchical clustering
- Advanced ML techniques:
 - Local Outlier Factor
 - Isolation Forests

Example Mahalanobis: (land) animal dataset

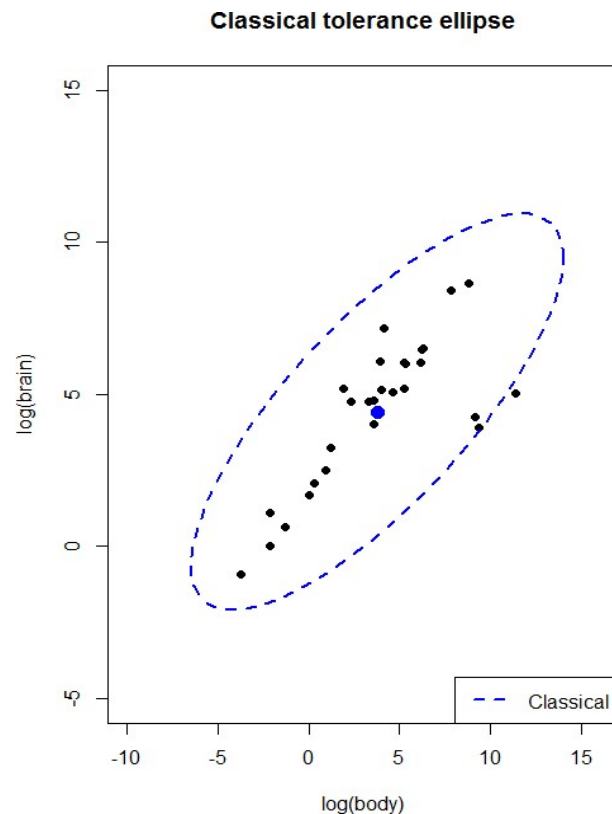


Table 7. Body and Brain Weight for 28 Animals

Index (i)	Species	Body Weight ^a (x_i)	Brain Weight ^b (y_i)
1	Mountain beaver	1.350	8.100
2	Cow	465.000	423.000
3	Gray wolf	36.330	119.500
4	Goat	27.660	115.000
5	Guinea pig	1.040	5.500
6	Diplodocus	11700.000	50.000
7	Asian elephant	2547.000	4603.000
8	Donkey	187.100	419.000
9	Horse	521.000	655.000
10	Potar monkey	10.000	115.000
11	Cat	3.300	25.600
12	Giraffe	529.000	680.000
13	Gorilla	207.000	406.000
14	Human	62.000	1320.000
15	African elephant	6654.000	5712.000
16	Triceratops	9400.000	70.000
17	Rhesus monkey	6.800	179.000
18	Kangaroo	35.000	56.000
19	Hamster	0.120	1.000
20	Mouse	0.023	0.400
21	Rabbit	2.500	12.100
22	Sheep	55.500	175.000
23	Jaguar	100.000	157.000
24	Chimpanzee	52.160	440.000
25	Brachiosaurus	87000.000	154.500
26	Rat	0.280	1.900
27	Mole	0.122	3.000
28	Pig	192.000	180.000

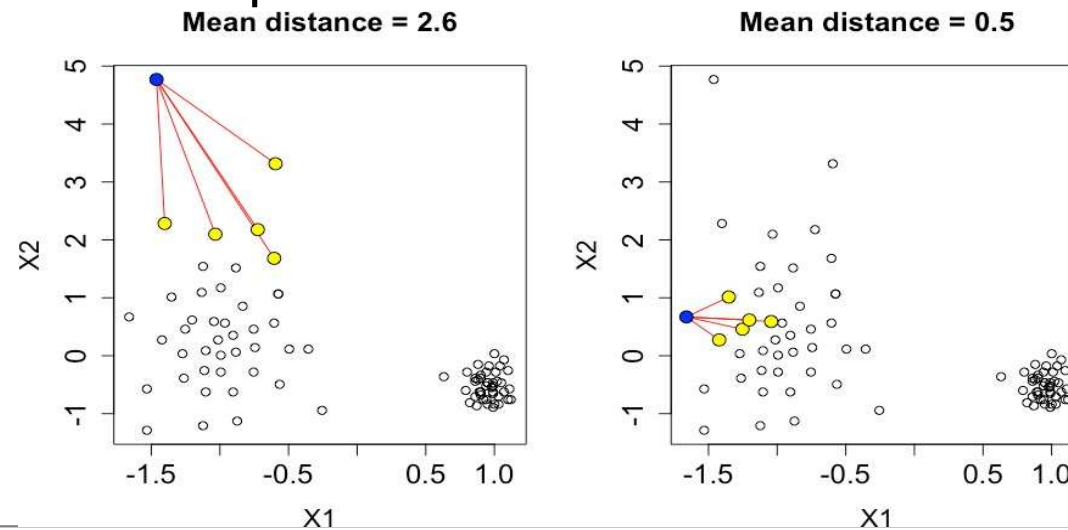
^aIn kilograms.

^bIn grams.

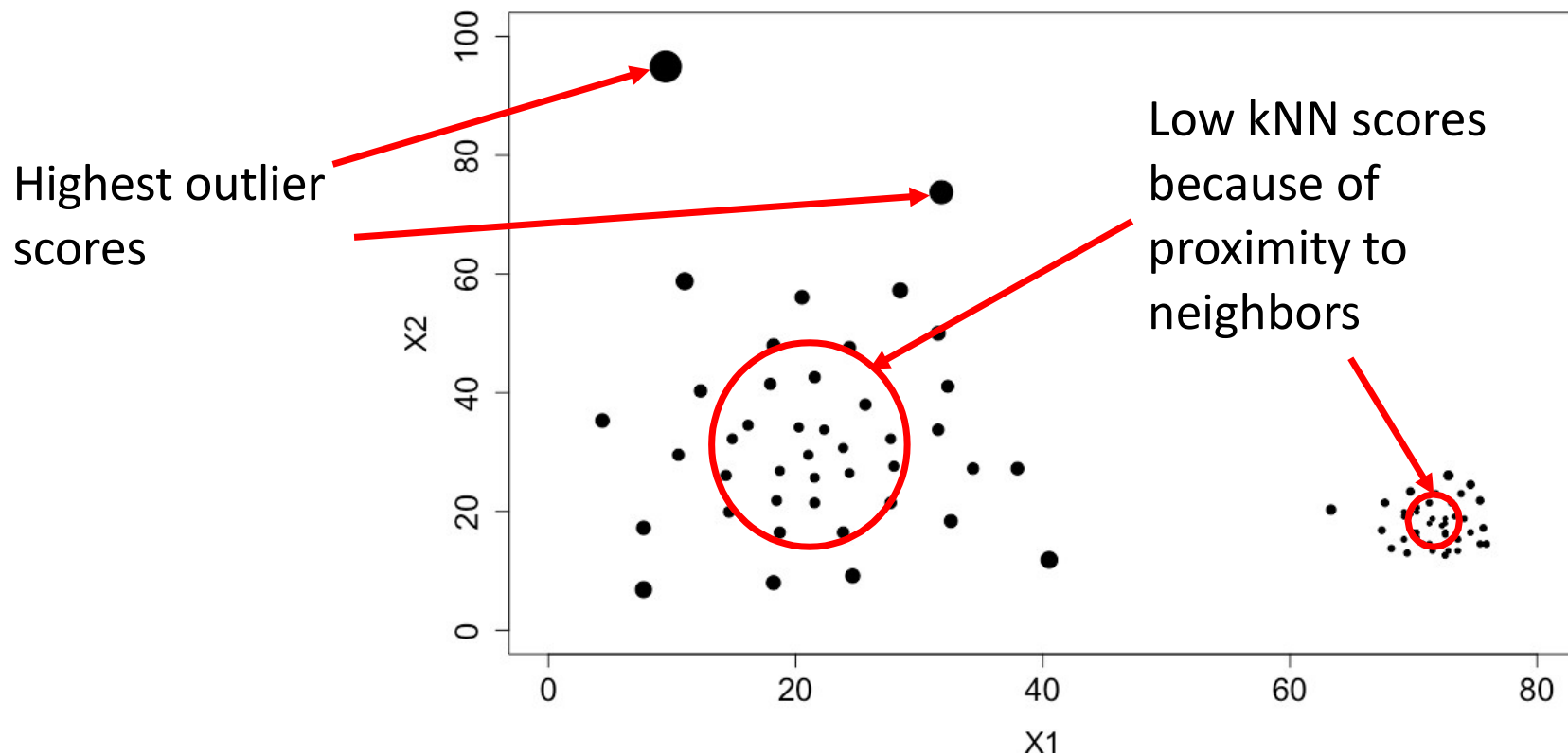
Source: Weisberg (1980) and Jerison (1973).

k-nearest neighbors (kNN) distance

- Anomalies typically lie far away from their neighbors (here: $k = 5$)
- kNN distance is an intuitive metric for measuring how isolated an observation is from neighboring observations
- Higher values indicate possible anomalies



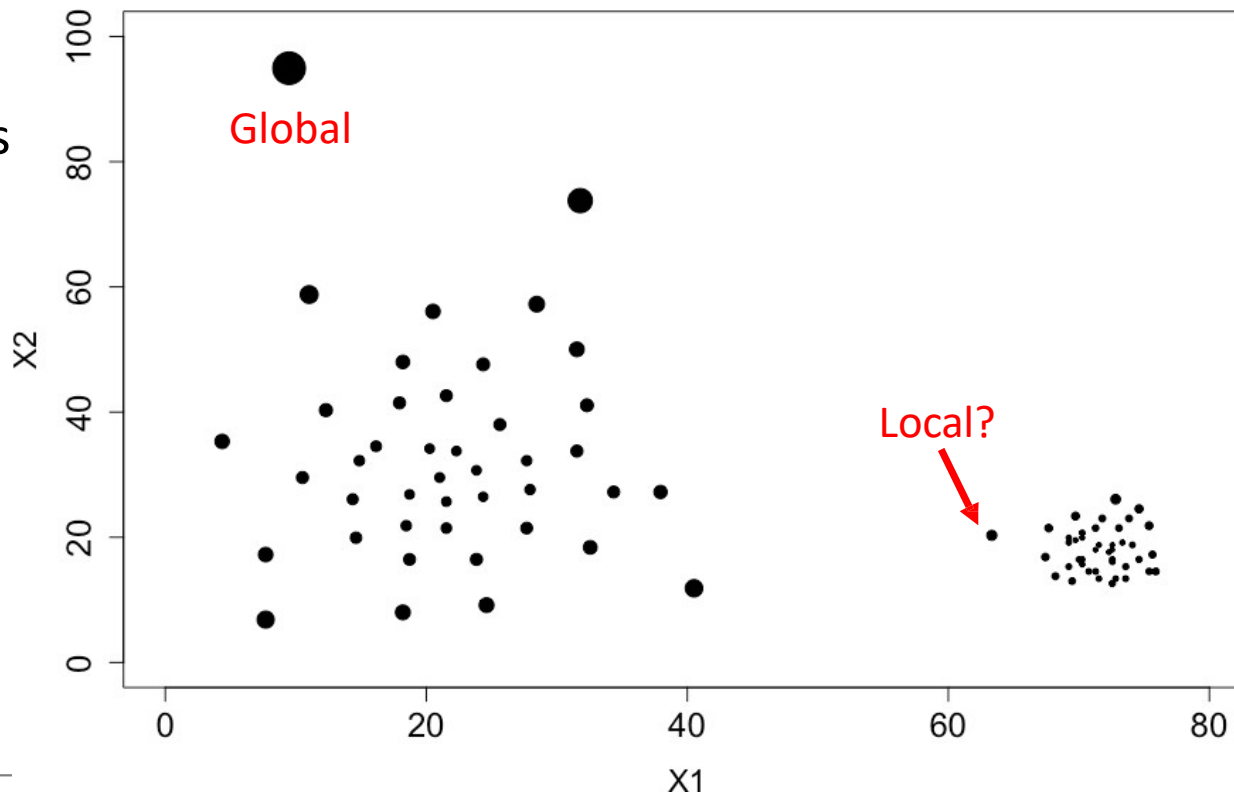
Visualization of kNN distance score



Local Outlier Factor (LOF)

Global versus local anomalies

kNN functions work well for detecting global anomalies



low kNN score, yet quite different from neighboring observations
 ➔ local anomalies can be better detected with **LOF**

Local Outlier Factor

- Local Outlier Factor: $LOF_k(A) = \frac{\sum_{X_j \in N_k(A)} LRD_k(X_j)}{|N_k(A)|} \times \frac{1}{LRD_k(A)}$
- LOF of A is average LRD of K neighbors of A to LRD of A
- $LOF(A) \sim 1$
 - similar densities so A is not an outlier
- $LOF(A) > 1$
 - LRD of A less than average LRD of neighbors so A likely to be outlier
- Generally: sort observations according to LOF and look at the biggest first

Local Outlier Factor

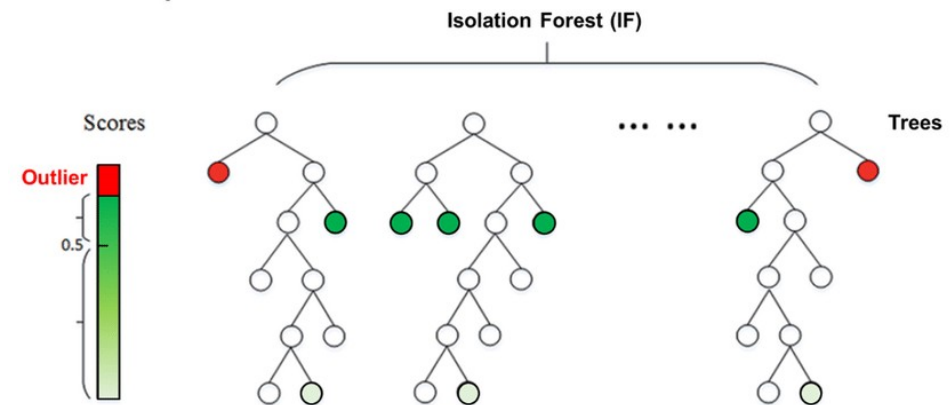
- Can effectively detect local outliers
- Need to choose K
- No threshold available

Isolation Forest (IF)

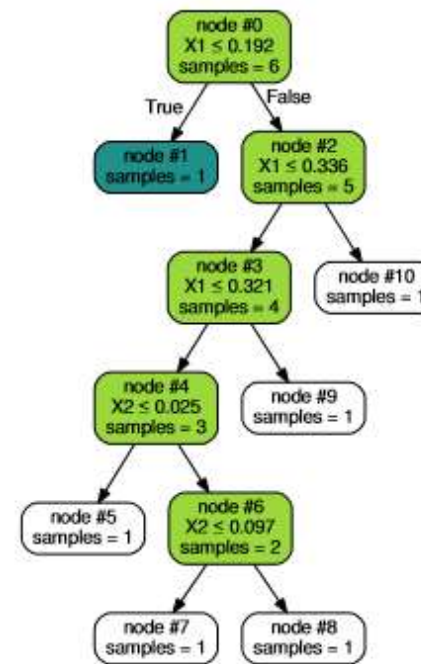
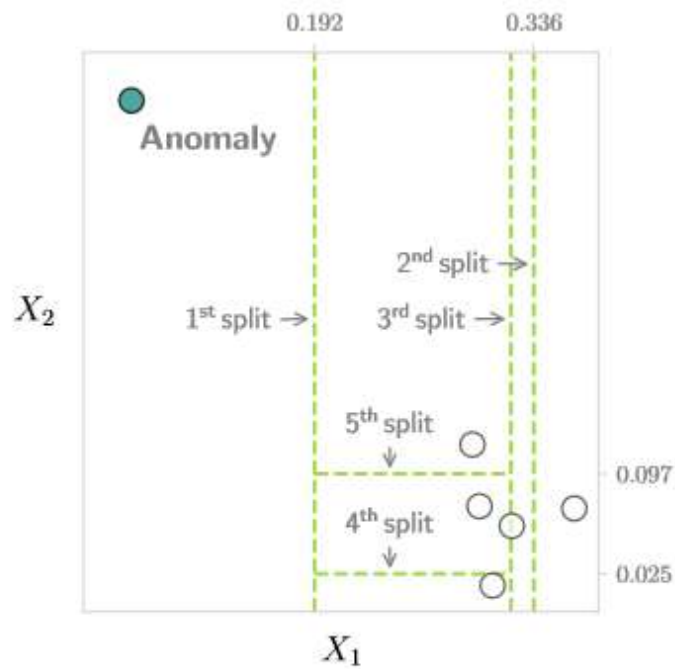
- Liu, Ting, Zhou, 2012
- Tree based approach to produce outlier scores
- **Idea:** points that are more easily separated from the other points are more deviant
- An isolation tree attempts to separate all of the points by randomly splitting the region in smaller pieces
- Random splitting continues until each point lies inside its own subregion or each subregion contains a maximum number of points

IF algorithm

- For $i=1$ to T (= number of trees in forest)
 - get sample of data
 - randomly pick variable
 - randomly pick value of that variable between its minimum and maximum
 - split data at that value
 - repeat until binary tree is complete
- Anomalies will be detected using only a few splits
- Normal observations need a lot of splits



Illustration



Evaluating anomaly detectors

- Without labels there is no accuracy; scores are rankings, not probabilities
- Work with the investigation capacity: the top-k alerts
 - Precision@k: share of the top-k alerts that turns out to be fraud after investigation
 - Confirmed cases become labels, which later allows a supervised model
- Preprocessing matters: scale features, transform skewed amounts, encode cyclical time
- Tooling
 - scikit-learn: EllipticEnvelope, LocalOutlierFactor, IsolationForest
 - PyOD: 60+ detectors with one interface (e.g. ECOD, COPOD)

Hands-on 4: anomaly detection

- Notebook 4 – anomaly detection ([open in Colab](#))
- Case: 30,492 card transactions, of which 492 frauds (labels only used for evaluation)
- Demo
 - Global vs. local anomalies on a toy example
 - z-score, Mahalanobis, kNN distance, LOF, Isolation Forest
 - Precision@k for an investigation budget
- Exercise 4A: tune the detectors to your budget
- Exercise 4B: more detectors from PyOD and an ensemble

What we observed in Notebook 4

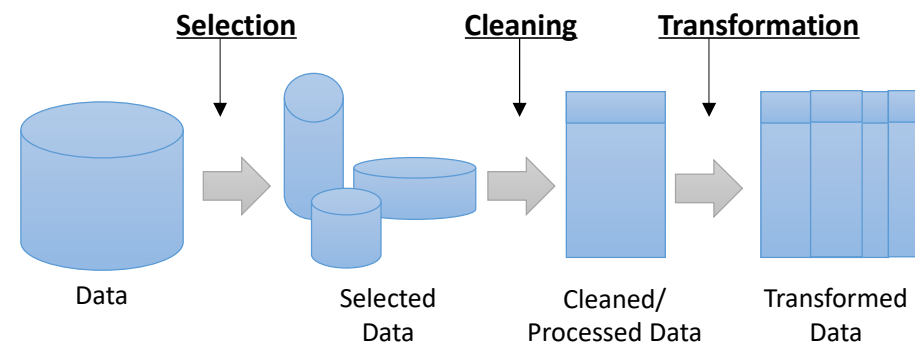
- The amount alone catches almost no fraud
- Multivariate methods do much better, the robust Mahalanobis distance best
- LOF does poorly with small k: frauds come in small, dense groups (masking)
- Detectors flag partly different transactions: combine them, and investigate

Detector	Precision@250	AUC
z-score of the amount	0.03	0.63
Mahalanobis	0.40	0.93
Mahalanobis (robust)	0.98	0.95
kNN distance (k = 20)	0.39	0.95
LOF (k = 20)	0.09	0.52
Isolation Forest	0.74	0.95

5. Conclusion

Conclusion

- Preprocessing involves a lot of different steps and checks, and starts with a good look at the data (EDA)
- The process is not a straight line: be ready to go back to step 1. Time consuming!
- Preprocessing depends on the technique to be used and the business question to be answered
- Everything before the model is part of the model: learn it on the training data and deploy it with the model
- Outliers are not always noise: sometimes they are exactly what you are looking for
- Keep in mind: can you apply your pre-processing to data coming in next month? How will it impact your results?



Two principles to remember

- Everything before the model is part of the model
 - Imputation values, scaling parameters, encodings and selected features are learned from data, just like model coefficients
 - They travel with the model into production
- A model can only learn from the world as it exists at prediction time
 - Every feature must be available at the moment the prediction is made
 - Violating this rule gives inflated performance and models that fail in deployment

A preprocessing checklist

- What exactly does the target reflect, and when is the prediction made?
- Which features would not be available at the prediction point?
- Where do I split, and randomly or in time?
- Which values are inconsistent, impossible or duplicated?
- Why are values missing: structural or random?
- Are the extremes errors, rare but valid values, or the signal I am looking for?
- Which transformations does my model need, and which of them are learned from data?
- Can the whole pipeline run unchanged on next month's data?

What you should know and what you should be able to do?

- You should know and understand the steps and requirements to select data for a data analytics project, including the prediction point and the risk of data leakage
- You should be able to carry out an exploratory data analysis and turn its findings into preprocessing decisions
- You should understand and be able to consistently apply data cleaning methods for missing values and outliers, taking into account a valid train-test setup
- You should know the main data transformation techniques, why they are relevant and oftentimes required to obtain good outcomes, and how to combine them in a pipeline
- You should understand the main anomaly detection techniques (Mahalanobis distance, kNN distance, LOF, Isolation Forest) and how to evaluate them